

Inheritance

Code reusability

java supports relationships

is A (parent child relationship/baseclass subclass/existingclass-derived class)

is A

has A dependencies and aggregation and composition

```
class Demo1
{
    String name="Hyder";
    int age=28;

    void disp()
    {
        System.out.println("Demo1 "+ age + name);
    }
}

class Demo2 extends Demo1
{
}
//Demo2 is child class/derived /sub
//Demo1 is parent/base/existing

public class LaunchIn1
{
    public static void main(String[] args) {

        Demo2 demo=new Demo2();
        demo.disp();

    }
}
```

```
//in this case demo2 is child/derived/sub  
//demo 1 is parent/base existing class
```

we developed a relation between classes using keyword extends

what it does it inherits all the property and behaviour of the parent class

Keypoints (Is a relationship)

- single inheritance is allowed that is one class can inherit another one class
- multi level inheritance is allowed
- object class is a parent of all class
- One parent class can have many child classes (hierarchical inheritance is allowed)
- Multiple inheritance is not allowed (ambiguity to compiler/diamond shape problem)
- cyclic inheritance is not allowed
- private members of a class does not participate in inheritance it is because to preserve the encapsulation feature of the java
- Constructor will not be participating in inheritance however parent class constructor will be getting called because of the super method in the constructor of the child class;;

Multilevel inheritance example

```
class Demo11 //extends Object  
{  
    //object properties and methods  
    String name="Hyder";  
    int age=28;  
  
    void disp()  
    {  
        System.out.println("Demo1 "+ age + name);  
    }  
}  
class Demo12 extends Demo11  
{  
  
}  
class Demo13 extends Demo12  
{  
  
}  
  
public class LaunchMultilevel {
```

```

public static void main(String[] args) {

    Demo13 d=new Demo13();
        d.disp();

}

}

```

in java if you do not extends any class by default it extends to object class whatever property of object is there will be inherited into the class even if we inherit another class from this class all the class things + object things got inherited from object class will be inherited too in second class

▼ herirechial inheritance

```

class Demo111
{
    String name="Hyder";
    int age=28;

    void disp()
    {
        System.out.println("Demo111 "+ age + name);
    }
}

class Demo112 extends Demo111
{
}

class Demo113 extends Demo111
{
}

class Demo114 extends Demo112
{
}

public class LaunchHierarchy {

    public static void main(String[] args) {

```

```

        // TODO Auto-generated method stub

    }

}

```

▼ CONSTRUCTOR GAME

```

//class Demo123 extends Demo321
//{

//}
//class Demo321 extends Demo123
//{

//}

class Parent11
{
    private String name;

    Parent11()
    {
        System.out.println("Parent Constructor");
    }
    void disp()
    {
        System.out.println("Parent ");
    }
}
class Child11 extends Parent11
{
    //Child11() //BEHIND THE SCENCE
    //{
    //    //super();
    //}
    void disp2()
    {
        //name="Hyder"; private memebrs will not participate in inheirtamce
    }
}
public class LaunchCyclic {

    public static void main(String[] args) {
        Child11 c=new Child11();
        c.disp();
    }
}

```

```
}
```

BEHIND THE SCENE WE KNOW THAT WHENEVER AN OBJECT IS CREATED A CONSTRUCTOR IS CALLED IN CONSTRUCTOR A SUPER/THIS METHOD which will be calling parent class constructor

memory map+inheritance

```
class Parentt
{
    int a, b;

    Parentt()
    {
        a=10;
        b=20;
        System.out.println("Parentt Const");
    }
    Parentt(int a, int b)
    {
        this.a=a;
        this.b=b;
        System.out.println("Parentt para Const");
    }
}

class Childd extends Parentt
{
    int x, y;

    Childd()
    {
        this(111, 222);
        x=100;
        y=200;
    }
    Childd(int x, int y)
    {
        this.x=x;
```

```

        this.y=y;
    }
    void disp()
    {
        System.out.println(a);
        System.out.println(b);
        System.out.println(x);
        System.out.println(y);
    }
}
public class LaunchConst {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Childd ch=new Childd();
        ch.disp();
        Childd ch1=new Childd(1000,2000);
        ch1.disp();

    }

}

```

Indepth explanation

- step 1 main method will be executed it will encounter the first line for creation of the object childd as soon as childd object created first it will extend and inherit all the properties of the parent class now the particular constructor will be called
- the second step is the first line of the constructor is super method by default if this() is not there super method will call the parent classs particular constructor this constructor will set the value of a=10,b=20 in the object childd a and b is already inherited in step 1 with default values that is 0 now after this parent constructor the value of a and b have been initialised
- parentt const will be printed now from wherever super method was called controlled will be going to that place from that place the value of x and y will be initialed after the ending of the constructor it will again go to the main method and lines of ahead will be executed

Consider below code of Counter.java file:

```
public class Counter {
    int count;

    private static void increment(Counter counter) {
        counter.count++;
    }

    public static void main(String [] args) {
        Counter c1 = new Counter();
        Counter c2 = c1;
        Counter c3 = null;
        c2.count = 1000;
        increment(c2);
    }
}
```

On executing Counter class, how many Counter objects are created in the memory?

- A. 1
- B. 2
- C. 3
- D. 4

Answer: A

Q>

Consider below code of Test.java file:

```
public class Test {
    public static void main(String[] args) {
        String res = "";
        loop: for(int i = 1; i <= 5; i++) { //Line n1
            switch(i) {
                case 1:
                    res += "UP ";
                    break;
                case 2:
                    res += "TO ";
                    break;
                case 3:
                    break;
                case 4:
                    res += "DATE";
                    break loop;
            }
        }
        System.out.println(res); //UP TO DATE
        System.out.println(String.join("-", res.split(" "))); //Line n2
    }
}
```

What will be the result of compiling and executing Test class?

```
i = 1
res = UP
i = 2
res = UP TO
i=3
res = UP TO
i=4
res = UP TO DATE
```

```
System.out.println(String.join("-",["UP","TO","DATE"]))//take the input from an
array and join with delimiter specified
System.out.println("UP-TO-DATE") //UP-TO-DATE
```

Q>

Consider below code of Test.java file:

```
public class Test {
    public static void main(String[] args) {
        String res = ""; //Line n1
        String [] arr = {"Dog", null, "Friendly"};
        for(String s : arr) { //Line n2
            res += String.join("-", s); //Line n3
        }
        System.out.println(res); //Line n4
    }
}
```

What will be the result of compiling and executing Test class?

- A. DogFriendly
- B. Dog-Friendly
- C. DognullFriendly
- D. Dog-null-Friendly
- E. NullPointerException is thrown
- F. CompileTimeError

```
res = ""
arr = {"Dog", null, "Friendly"}
s = Dog
    res = "" + String.join("-", "Dog")
        = "" + Dog
        = "Dog"
s = null
    res = "Dog"+ String.join("-", null)
        = "Dognull"

s = Friendly
    res = "Dognull" + String.join("-", "Friendly")
        = "Dognull"+"Friendly"
    res = "DognullFriendly"
```

Output: C

Q>

Given code

```
public static void main(String[] args) {
    String[][] chs = new String[2][];
    chs[0] = new String[2]; chs[1] = new String[5]; int i=97;
    for (int a=0; a<chs.length; a++) {
        for (int b=0; b<chs[a].length; b++) {
            chs[a][b] = " "+i;
            i++;
        }
    }
    for (String[] ca: chs) {
        for (String c: ca) {
            System.out.print(c + " ");
        }
        System.out.println();
    }
}
```

```

    }
}
A. 97 98
    99 100 null null null
B. 97 98
    99 100 101 102 103
C. Compilation fails.
D. NullPointerException is thrown at runtime.
E. ArrayIndexOutOfBoundsException is thrown at runtime.

```

Answer: A

Q>

```

public static void main(String... args) {
    String ta="A ";
    ta = ta.concat("B ");
    String tb="C ";
    ta = ta.concat(tb);
    ta.replace('C', 'D');
    ta=ta.concat(tb);
    System.out.println(ta);
}

```

- A. A B C D
- B. A C D
- C. A B C C
- D. A B D
- E. A B D C
- F. CompileTimeError
- G. Some Problem at the runtime by JVM.

```

ta = "A B C C "
tb = "C "

```

Answer: C

Q>

Consider below code of Test.java file:

```

public class Test {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("B"); //Line n1
        sb.append(sb.append("A")); //Line n2
        System.out.println(sb); //Line n3
    }
}

```

What will be the result of compiling and executing Test class?

- A. B
- B. BA
- C. AB
- D. BAB
- E. ABA
- F. ABAB
- G. BABA
- H. ABBA
- I. CompilationError at line 2

```

sb = "BA"
"BA".append("BA");
sb = "BABA"

```

```
Q>
public class Test {
    public static void main(String[] args) {
        String [] arr = {"1st", "2nd", "3rd", "4th", "5th"};
        String place = "faraway";
        System.out.println(arr[place.indexOf("a", 3)]); //Line n1
    }
}
```

What will be the result of compiling and executing Test class?

- A. 1st
- B. 3rd
- C. 4th
- D. 5th
- E. 2nd
- F. RuntimeException
- G. Compiletime Error at line n-1

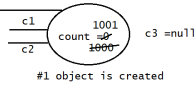
```
arr[0] = 1st
arr[1]= 2nd
arr[2] = 3rd
arr[3] = 4th
arr[4] = 5th
place = "faraway"
System.out.println( arr[3]);
```

Answer: C

```
private static void increment(Counter counter) {
    counter.count++;
}
```

```
Counter c1 = new Counter();
Counter c2 = c1;
Counter c3 = null;
c2.count = 1000;
increment(c2);
```

#1 object is created



```
String[][] chs = new String[2][];
chs[0] = new String[2];
chs[1] = new String[5];
int i=97;
for (int a=0; a<chs.length;a++){
    for (int b=0;b<chs[a].length;b++){
        chs[a][b] = " "+i;
        i++;
    }
}
for (String[] ca:chs){
    for (String c: ca ){
        System.out.print(c + " ");
    }
    System.out.println();
}
```

